routes\b2bOrder.ts

b2bOrder.ts

» javascript.express.security.audit.express-detect-notevil-usage.express-detect-notevil-usage

« Blocking »

Detected usage of the `notevil` package, which is unmaintained and has vulnerabilities. Using any sort of `eval()` functionality can be very dangerous, but if you must, the `eval` package is an up to date alternative. Be sure that only trusted input reaches an `eval()` function.
Details: https://sg.run/W70E

```
23 vm.runInContext('safeEval(orderLinesData)', sandbox, { timeout: 2000 })
```

- Exact Match
- Correct Risk, Wrong Class
- Close Match
- No Match

Semgrep finding

A03:2025 – Software Supply Chain Failures
CWE-1104 Use of Unmaintained Third-Party Components

routes\b2bOrder.ts

```
import { eval as safeEval } from 'notevil'; vm.runInContext('safeEval(orderLinesData)', sandbox, { timeout: 2000 }); ch
```

A05:2025 - Injection

Confidence: 0.9

Risk Summary: The code is vulnerable to injection attacks as it uses the vm module to execute user-controlled input and directly evaluates user input. This could lead to remote code execution (RCE) and other security issues.

Recommendation: Validate and sanitize all user input to prevent code injection. Use a whitelist approach to only allow expected input formats. Use a safer evaluation method, such as a JSON parser, or validate and sanitize the user input before passing it to the vm module.

LLM Analysis, with Chunking

routes\b2bOrder.ts

```
vm.runInContext('safeEval(orderLinesData)', sandbox, { timeout: 2000 })
```

A05:2025 - Injection

Confidence: 0.9

Risk Summary: The code is vulnerable to code injection attacks. An attacker could inject malicious code, potentially leading to remote code execution. The use of the 'safeEval' function does not guarantee safety, as it can still be exploited.

Recommendation: Avoid using 'eval' or 'safeEval' functions. Instead, use a safer method to parse and execute the orderLinesData, such as using a JSON parser or a templating engine.

LLM Analysis, without Chunking

routes\captcha.ts

```
captcha.ts
❯❯ javascript.browser.security.eval-detected.eval-detected
   ❮❮ Blocking ❯❯
   Detected the use of eval(). eval() can be dangerous if used to evaluate dynamic content. If this
   content can be input from outside the program, this may be a code injection vulnerability. Ensure
   evaluated content is not definable by external sources.
   Details: https://sg.run/7ope

   23 ┆ const answer = eval(expression).toString() // eslint-disable-line no-eval
```

- Exact Match
- Correct Risk, Wrong Class
- Close Match
- No Match

Semgrep finding

A05:2025 – Injection
CWE-95 Improper Neutralization of Directives in Dynamically Evaluated Code ('Eval Injection')

routes\captcha.ts

```
const answer = eval(expression).toString()
```

A05:2025 - Injection

Confidence: 0.9

Risk Summary: The use of eval() function can lead to code injection attacks, potentially allowing an attacker to execute malicious code. This can result in unauthorized access, data tampering, or other security breaches.

Recommendation: Instead of using eval(), consider using a safer method to evaluate the mathematical expression, such as using a library or writing a custom function to calculate the result.

LLM Analysis, with Chunking

routes\captcha.ts

```
const answer = eval(expression).toString()
```

A05:2025 - Injection

Confidence: 0.9

Risk Summary: The use of eval() function can lead to code injection attacks. An attacker could potentially inject malicious code, leading to security vulnerabilities.

Recommendation: Use a safer method to evaluate the expression, such as using a library or writing a custom function to calculate the result.

LLM Analysis, without Chunking

routes\chatbot.ts

```
chatbot.ts
❯❯ javascript.express.security.injection.raw-html-format.raw-html-format
  ❮❮ Blocking ❯❯
  User data flows into the host portion of this manually-constructed HTML. This can introduce a Cross-
  Site-Scripting (XSS) vulnerability if this comes from user-provided input. Consider using a
  sanitization library such as DOMPurify to sanitize the HTML within.
  Details: https://sg.run/5DO3

  197 ¦ body: bot.training.state ? bot.greet(`${user.id}`) :
      `${config.get<string>('application.chatBot.name')} isn't ready at the moment, please wait
      while I set things up`
```

- Exact Match
- Correct Risk, Wrong Class
- Close Match
- No Match

Semgrep finding

A05:2025 – Injection
CWE-79 Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')

LLM Analysis, with Chunking

LLM Analysis, without Chunking

routes\fileServer.ts

```
fileServer.ts
❯❯ javascript.express.security.audit.express-res-sendfile.express-res-sendfile
   ❮❮ Blocking ❯❯
   The application processes user-input, this is passed to res.sendFile which can allow an attacker to
   arbitrarily read files on the system through path traversal. It is recommended to perform input
   validation in addition to canonicalizing the path. This allows you to validate the path against the
   intended directory it should be accessing.
   Details: https://sg.run/7DJk

   33 ┆ res.sendFile(path.resolve('ftp/', file))
```

- Exact Match
- Correct Risk, Wrong Class
- Close Match
- No Match

Semgrep finding

A01:2025 – Broken Access Control
CWE-22 Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')

LLM Analysis, with Chunking

routes\fileServer.ts

```
res.sendFile(path.resolve('ftp/', file))
```

A05:2025 - Injection

Confidence: 0.8

Risk Summary: The code is vulnerable to a path traversal attack, allowing an attacker to access files outside the intended directory. This could lead to sensitive data exposure or code execution.

Recommendation: Use a whitelist approach to validate the file path and ensure it's within the intended directory. Consider using a library like `path-normalize` to normalize the file path.

LLM Analysis, without Chunking

routes\fileUpload.ts

```
fileUpload.ts
❯❯ javascript.express.security.audit.express-libxml-vm-noent.express-libxml-vm-noent
  ❮❮ Blocking ❯❯
  Detected use of parseXml() function with the `noent` field set to `true`. This can lead to an XML
  External Entities (XXE) attack if untrusted data is passed into it.
  Details: https://sg.run/n8Ag

  83 ┆ const xmlDoc = vm.runInContext('libxml.parseXml(data, { noblanks: true, noent: true,
     nocdata: true })', sandbox, { timeout: 2000 })
```

- Exact Match
- Correct Risk, Wrong Class
- Close Match
- No Match

Semgrep finding

A02:2025 – Security Misconfiguration
CWE-611 Improper Restriction of XML External Entity Reference

routes\fileUpload.ts

```
import vm from 'node:vm'; const absolutePath = path.resolve('uploads/complaints/' + fileName); const xmlDoc = vm.runI
```

A05:2025 - Injection

Confidence: 0.9

Risk Summary: The code is vulnerable to XML External Entity (XXE) attacks, YAML injection attacks, directory traversal attacks, and code injection vulnerabilities.

Recommendation: Use a secure XML parsing library that is not vulnerable to XXE attacks, such as xml2js or fast-xml-parser. Also, validate and sanitize the uploaded XML file to prevent malicious entities. Use a safe YAML parsing library that prevents code injection, such as js-yaml with the `safeLoad` function, and validate user input to ensure it conforms to expected formats. Validate and sanitize the file paths extracted from the zip file to prevent directory traversal attacks. Use a library like `path.normalize()` to normalize the file paths and prevent attacks.

LLM Analysis, with Chunking

```
const xmlDoc = vm.runInContext('libxml.parseXml(data, { noblanks: true, noent: true, nocdata: true })', sandbox, { tim
```

A04:2025 - Cryptographic Failures

Confidence: 0.9

Risk Summary: The code is vulnerable to XML External Entity (XXE) attacks. An attacker can upload a malicious XML file that can read sensitive data from the server.

Recommendation: Use a secure XML parser that is not vulnerable to XXE attacks. Validate and sanitize the XML data before parsing it.

LLM Analysis, without Chunking

routes\login.ts

```
>>> javascript.sequelize.security.audit.sequelize-injection-express.express-sequelize-injection
  << Blocking >>
  Detected a sequelize statement that is tainted by user-input. This could lead to SQL injection if
  the variable is user-controlled and is not properly sanitized. In order to prevent SQL injection, it
  is recommended to use parameterized queries or prepared statements.
  Details: https://sg.run/gjoe

  34 ┆ models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email || ''}' AND
     password = '${security.hash(req.body.password || '')}' AND deletedAt IS NULL`, { model:
     UserModel, plain: true }) // vuln-code-snippet vuln-line loginAdminChallenge
     loginBenderChallenge loginJimChallenge
```

- Exact Match
- Correct Risk, Wrong Class
- Close Match
- No Match

Semgrep finding

A05:2025 – Injection
CWE-89 Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')

```
models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email || ''}' AND password = '${security.hash
```

A05:2025 - Injection

Confidence: 0.9

Risk Summary: The code is vulnerable to SQL injection attacks. An attacker can inject malicious SQL code by manipulating the email or password fields.

Recommendation: Use parameterized queries or prepared statements to prevent SQL injection attacks.

LLM Analysis, with Chunking

routes\login.ts

```
models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email || ''}' AND password = '${security.hash(
```

A05:2025 - Injection

Confidence: 0.9

Risk Summary: The code is vulnerable to SQL injection attacks. An attacker can inject malicious SQL code by manipulating the email or password fields.

Recommendation: Use parameterized queries or prepared statements to prevent SQL injection attacks.

LLM Analysis, without Chunking

routes\redirect.ts

redirect.ts

❯❯ javascript.express.open-redirect-deepsemgrep.open-redirect-deepsemgrep

❮❮ Blocking ❯❯

The application builds a URL using user-controlled input which can lead to an open redirect vulnerability. An attacker can manipulate the URL and redirect users to an arbitrary domain. Open redirect vulnerabilities can lead to issues such as Cross-site scripting (XSS) or redirecting to a malicious domain for activities such as phishing to capture users' credentials. To prevent this vulnerability perform strict input validation of the domain against an allowlist of approved domains. Notify a user in your application that they are leaving the website. Display a domain where they are redirected to the user. A user can then either accept or deny the redirect to an untrusted site.

Details: https://sg.run/BDbW

```
19 | res.redirect(toUrl)
```

- Exact Match
- Correct Risk, Wrong Class
- Close Match
- No Match

Semgrep finding

A01:2025 – Broken Access Control

CWE-601 URL Redirection to Untrusted Site ('Open Redirect')

routes\redirect.ts

```
import * as security from '../lib/insecurity'; return toUrl === 'https://explorer.dash.org/address/Xr556RzuwX6hg5EGpkyb
for (const allowedUrl of security.redirectAllowlist) {
unintended = unintended && !utils.startsWith(toUrl, allowedUrl)
}
```

A02:2025 - Security Misconfiguration

Confidence: 0.8

Risk Summary: The code imports a module named 'insecurity' which may indicate the presence of intentional security vulnerabilities. This could lead to various security issues, including data breaches and unauthorized access. The function isUnintendedRedirect does not properly validate URLs, potentially allowing unauthorized redirects. This could lead to phishing or other attacks. The function only checks if the URL does not start with any of the allowed URLs, but it does not check for other potential issues.

Recommendation: Implement a more robust URL validation mechanism, such as using a whitelist of allowed URLs and checking for any potential redirects or URL manipulations. Remove or refactor the 'insecurity' module to ensure it does not introduce security vulnerabilities.

LLM Analysis, with Chunking

routes\redirect.ts

```
res.redirect(toUrl)
```

A05:2025 - Injection

Confidence: 0.9

Risk Summary: The code is vulnerable to an open redirect attack. An attacker can manipulate the 'toUrl' parameter to redirect users to a malicious website. This can lead to phishing attacks or other malicious activities.

Recommendation: Validate and sanitize the 'toUrl' parameter to ensure it only redirects to trusted and expected URLs.

LLM Analysis, without Chunking

routes\showProductReviews.ts

showProductReviews.ts

❯❯❯❯ javascript.express.mongodb.express-mongo-nosqli.express-mongo-nosqli

❮❮ Blocking ❯❯

Detected a `../data/mongodb` statement that comes from a `req` argument. This could lead to NoSQL injection if the variable is user-controlled and is not properly sanitized. Be sure to properly sanitize the data if you absolutely must pass request data into a mongo query.
Details: https://sg.run/Qqxq

```
36 db.reviewsCollection.find({ $where: 'this.product == ' + id }).then((reviews: Review[])
   => {
```

- Exact Match
- Correct Risk, Wrong Class
- Close Match
- No Match

Semgrep finding

A05:2025 – Injection
CWE-943 Improper Neutralization of Special Elements in Data Query Logic

```
Number(req.params.id) : utils.trunc(req.params.id, 40); db.reviewsCollection.find({ $where: 'this.product == ' + id })
```

A05:2025 - Injection

Confidence: 0.9

Risk Summary: The code is vulnerable to NoSQL injection attacks. An attacker could inject malicious NoSQL code, potentially leading to data breaches or unauthorized access.

Recommendation: Use parameterized queries or the $eq operator to prevent NoSQL injection attacks. For example: db.reviewsCollection.find({ product: id }). Use a whitelist approach to validate user input and ensure that only expected data types are accepted. Consider using a library like Joi or express-validator to validate user input.

LLM Analysis, with Chunking

routes\showProductReviews.ts

```
db.reviewsCollection.find({ $where: 'this.product == ' + id })
```

A05:2025 - Injection

Confidence: 0.8

Risk Summary: The code is vulnerable to NoSQL injection attacks. An attacker could inject malicious NoSQL code, potentially leading to data breaches or unauthorized access.

Recommendation: Use parameterized queries or prepared statements to prevent NoSQL injection attacks. For example, use the $eq operator to compare the product field with the id variable.

LLM Analysis, without Chunking

routes\updateUserProfile.ts (1)

updateUserProfile.ts

javascript.express.web.cookies-default-express.cookies-default-express

Blocking

Having default cookie settings is insecure because they often lack critical security attributes, leaving cookies vulnerable to various attacks like CSRF or XSS. Always configure cookies with security attributes: `HttpOnly`, `Secure`, `SameSite`.
Details: https://sg.run/JDG5Z

```
40 res.cookie('token', updatedToken)
```

- Exact Match
- Correct Risk, Wrong Class
- Close Match
- No Match

Semgrep finding

A02:2025 – Security Misconfiguration
CWE-1004 Sensitive Cookie Without 'HttpOnly' Flag
CWE-614 Sensitive Cookie in HTTPS Session Without 'Secure' Attribute
CWE-1275 Sensitive Cookie with Improper SameSite Attribute

routes\updateUserProfile.ts

```
const loggedInUser = security.authenticatedUsers.get(req.cookies.token)
```

A07:2025 - Authentication Failures

Confidence: 0.8

Risk Summary: The code uses a token from the request cookies to authenticate the user. This approach is vulnerable to token hijacking and replay attacks. An attacker could steal the token and use it to impersonate the user.

Recommendation: Use a secure method for authentication, such as using a secure cookie with the 'httpOnly' and 'secure' flags, or using a token-based authentication mechanism with proper token validation and renewal.

LLM Analysis, with Chunking

routes/updateUserProfile.ts

```
const loggedInUser = security.authenticatedUsers.get(req.cookies.token)
```

A07:2025 - Authentication Failures

Confidence: 0.8

Risk Summary: The code uses a token stored in a cookie to authenticate users. This approach is vulnerable to token theft and tampering. An attacker could steal the token and use it to access the user's account.

Recommendation: Use a secure authentication mechanism, such as JSON Web Tokens (JWT) or session-based authentication, and store the token securely on the client-side, such as using the Secure and HttpOnly flags for cookies.

LLM Analysis, without Chunking

routes\updateUserProfile.ts (2)

```
❯❯ javascript.express.session-fixation.session-fixation
   ❮❮ Blocking ❯❯
   Detected `req` argument which enters `res.cookie`, this can lead to session fixation vulnerabilities
   if an attacker can control the cookie value. This vulnerability can lead to unauthorized access to
   accounts, and in some esoteric cases, Cross-Site-Scripting (XSS). Users should not be able to
   influence cookies directly, for session cookies, they should be generated securely using an approved
   session management library. If the cookie does need to be set by a user, consider using an allow-
   list based approach to restrict the cookies which can be set.
   Details: https://sg.run/oqDv

   40 ┆ res.cookie('token', updatedToken)
```

- Exact Match
- Correct Risk, Wrong Class
- Close Match
- No Match

Semgrep finding

A07:2025 – Authentication Failures
CWE-384 Session Fixation

routes\updateUserProfile.ts

```
const loggedInUser = security.authenticatedUsers.get(req.cookies.token)
```

A07:2025 - Authentication Failures

Confidence: 0.8

Risk Summary: The code uses a token from the request cookies to authenticate the user. This approach is vulnerable to token hijacking and replay attacks. An attacker could steal the token and use it to impersonate the user.

Recommendation: Use a secure method for authentication, such as using a secure cookie with the 'httpOnly' and 'secure' flags, or using a token-based authentication mechanism with proper token validation and renewal.

LLM Analysis, with Chunking

routes/updateUserProfile.ts

```
const loggedInUser = security.authenticatedUsers.get(req.cookies.token)
```

A07:2025 - Authentication Failures

Confidence: 0.8

Risk Summary: The code uses a token stored in a cookie to authenticate users. This approach is vulnerable to token theft and tampering. An attacker could steal the token and use it to access the user's account.

Recommendation: Use a secure authentication mechanism, such as JSON Web Tokens (JWT) or session-based authentication, and store the token securely on the client-side, such as using the Secure and HttpOnly flags for cookies.

LLM Analysis, without Chunking

routes\userProfile.ts

```
userProfile.ts
❯❯ javascript.browser.security.eval-detected.eval-detected
  ❮❮ Blocking ❯❯
  Detected the use of eval(). eval() can be dangerous if used to evaluate dynamic content. If this
  content can be input from outside the program, this may be a code injection vulnerability. Ensure
  evaluated content is not definable by external sources.
  Details: https://sg.run/7ope

  62 ┆ username = eval(code) // eslint-disable-line no-eval
```

- Exact Match
- Correct Risk, Wrong Class
- Close Match
- No Match

Semgrep finding

A05:2025 – Injection
CWE-94 Improper Control of Generation of Code ('Code Injection')

```
let username = user.username; username = eval(code); template = template.replace(/_username_/g, username); utils.contai
```

A05:2025 - Injection

Confidence: 0.9

Risk Summary: The code is vulnerable to injection attacks as it directly assigns user input (username) to a variable without proper validation or sanitization. The eval() function can evaluate any JavaScript code, which makes it possible for an attacker to inject malicious code. The code is also vulnerable to template injection and XSS attacks.

Recommendation: Validate and sanitize the username input using a library like DOMPurify or a similar solution to prevent injection attacks. Use a safer alternative to eval(), such as a templating engine or a JSON parser, to prevent code injection attacks. Use a templating engine that escapes user input by default, or manually escape the username variable before passing it to the template. Validate and sanitize user input to prevent XSS attacks.

LLM Analysis, with Chunking

routes/userProfile.ts

```
username = eval(code)
```

A05:2025 - Injection

Confidence: 0.9

Risk Summary: The code is vulnerable to code injection via the username field. An attacker could inject malicious code, potentially leading to security breaches.

Recommendation: Use a safer method to evaluate the username, such as using a templating engine or escaping the input.

LLM Analysis, without Chunking

routes\videoHandler.ts

```
videoHandler.ts
❯❯ javascript.lang.security.audit.unknown-value-with-script-tag.unknown-value-with-script-tag
  ❮❮ Blocking ❯❯

  Cannot determine what 'subs' is and it is used with a '<script>' tag. This could be susceptible to
  cross-site scripting (XSS). Ensure 'subs' is not externally controlled, or sanitize this data.
  Details: https://sg.run/1Zy1

  58┆ challengeUtils.solveIf(challenges.videoXssChallenge, () => { return utils.contains(subs,
    '</script><script>alert(`xss`)</script>') })
  ⋮┆----------------------------------------
  71┆ compiledTemplate = compiledTemplate.replace('<script id="subtitle"></script>', '<script
    id="subtitle" type="text/vtt" data-label="English" data-lang="en">' + subs + '</script>')
```

- Exact Match
- Correct Risk, Wrong Class
- Close Match
- No Match

Semgrep finding

A05:2025 – Injection
CWE-79 Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')

```
challengeUtils.solveIf(challenges.videoXssChallenge, () => { return utils.contains(subs, '</script><script>alert(`xss`)
```

A05:2025 - Injection

Confidence: 0.9

Risk Summary: The code is vulnerable to Cross-Site Scripting (XSS) attacks, template injection attacks, and path traversal attacks. An attacker could inject malicious scripts, templates, or file paths, potentially leading to unauthorized access or data theft.

Recommendation: Validate and sanitize all user input to prevent XSS attacks. Use a templating engine that automatically escapes user input, or use a library like DOMPurify to sanitize the input. Additionally, use a whitelist to validate file names and ensure they only contain allowed characters. Use a secure method to read files, such as using a library that sanitizes file paths.

LLM Analysis, with Chunking

routes\videoHandler.ts

```
challengeUtils.solveIf(challenges.videoXssChallenge, () => { return utils.contains(subs, '</script><script>alert(`xss`)
```

A05:2025 - Injection

Confidence: 0.8

Risk Summary: The code is vulnerable to a Cross-Site Scripting (XSS) attack. The 'subs' variable is not properly sanitized, allowing an attacker to inject malicious code.

Recommendation: Properly sanitize the 'subs' variable using a library like DOMPurify to prevent XSS attacks.

LLM Analysis, without Chunking

12 vulnerabilities from the baseline, compared to LLM findings

With Chunking

- 5 Exact Match
- 1 Correct Risk, Wrong Class
- 4 Close Match
- 2 No Match

Without Chunking

- 5 Exact Match
- 3 Correct Risk, Wrong Class
- 3 Close Match
- 1 No Match